REXUS 38

STAR – D Experiment

Software Development

CONOPS – Concept of Operations Document

Version: 3.0
Issue date: 15/02/2026
Contributor: Alexandre Rollet
(Software engineer)

[PAGE INTENTIONALLY LEFT BLANK]

Table of Contents

1. **Introduction**
   1.1 Abstract
   1.2 Scope
   1.3 References
   1.4 Assumptions
2. **Mission overview relevant to software operations**
   2.1 Mission objectives
   2.2 Mission phases
   2.3 Success criteria
3. **Software operational concept**
   3.1 Concept of operations
   3.2 Software modes and state transitions
      3.2.1 Modes
      3.2.2 Operational scenarios
   3.3 Interfaces
4. **Data flow and management**
   4.1 Sources and acquisition
   4.2 Processing and storage, data flow
   4.3 Data output and telemetry
5. **Software constraints and requirements overview**
   5.1 Physical and environmental constraints
   5.2 Operational constraints
6. **Verification and validation overview**
   6.1 Verification concept
   6.2 Validation concept
7. **Abbreviations and notes**

1 – Introduction

1.1 Abstract

This document serves as a general description of the mission's software systems and development. It will give an overview of how the software will be designed and the requirements that will be enforced to ensure its robustness. The software will be explained through its role in the experiment, its functionality, expected operation times, and the framework used to validate its model.

1.2 Scope

This document is to be read as the project's design manual. Technical and project management elements will be presented to explain expected core functionalities and work development. General specifications that will need to be followed will be given. The document answers the question of *what must happen, when and why.* However, the document does not give specific technical details on software performance or reliable numerical data. Subsequent documents will explain precise requirements, software architecture and in-depth interface specifications.

1.3 References

- European Cooperation for Space Standardization (ECSS), ECSS-E-ST-10C Rev.1 – "Software Engineering", 15 February 2017.

The above reference was used as a guideline for developing this document. The structure and content, however, do not strictly adhere to the official standard.

- STAR-D Electronic Systems Proposal document
- RX_UserManual_v7-17_26Nov21

1.4 Assumptions

It is assumed that:

- all electronic components are operational or, at a minimum, meet the specified minimum performance requirements. External

temperatures during the flight have no effect on electronic components
- no mechanical errors occur during the mission
- the sounding rocket performs a nominal flight
- telemetry loss does not affect onboard autonomy (however, details on this scenario will be given later in the scope of studying operational outcomes and risk management)

This allows potential software errors to be identified and evaluated within their respective domain.

2 – Mission overview relevant to software operations

2.1 Mission objectives

The software's objective is to enable semi-autonomous execution of the experiment for the whole duration of the mission. The software is responsible for data logging, temperature control, and telemetry transmission. Software-related mission objectives are as follows:

- The software autonomously and continuously monitors the temperature inside the experiment module, and enforces temperature corrections
- To collect, format and store camera data that records the microscope's view of the biology experiment
- To correctly format and transmit telemetry data with a fixed and limited packet size
- Acquire, manage, and preserve experimental data
- Provide experiment health monitoring through the telemetry downlink
- Detect and respond to off-nominal conditions
- Ensure modular and fault-tolerant architecture
- Coordinate experiment sequencing and operational modes

2.2 Mission phases

Phase	Explanation	Action(s)	Possible actions
Pre-launch, setup	Software upload to EM and startup	Startup and system test	Multiple test startups
Launch countdown	Countdown clock begins	SODS shortly before LO, receive telemetry, temperature control	Telecommands to start/stop SODS, activate RS, check system health
Launch	Rocket goes into flight	LO, higher frequency of metadata storage	
Ascent	Rocket ascends		
Motor burnout	Coast, microgravity	SOE on	
Descent	End of microgravity	SOE off, SODS off shortly before EM power cutoff	Lower frequency of metadata storage
Recovery	Recovery of data from storage card	Upload storage data into external computer	

Fig. 1 Mission phases overview (simplified)

2.3 Success criteria

Software success is defined by its ability to function semi-autonomously throughout the mission, to transition correctly between modes, to collect and preserve all crucial data from the experiment and sensors, and to continuously maintain the EM at a strict temperature level. Each success criterion will be evaluated and linked to a specific software requirement in the *Software Requirements Specification* document. In this section, each success criterion is given verification evidence that is traceable and a criticality factor.

Traceability will be extensively applied in all development phases, and future documentation will ensure that specific and testable features are given to eliminate the risk of neglecting any potential errors.

ID	Success criterion	Operational phase	Verification evidence	Criticality
SC-01	Software can be uploaded to hardware without errors	Setup	No crash, successful upload observed on electronics RX LEDs	Critical
SC-02	Software boots nominally after power-on	Setup	Boot logs, electronics LED status	High
SC-03	Transition to flight mode during launch detection/uplink signal	Countdown, launch	Telemetry logs, downlink observation	High
SC-04	Software manages to maintain stable module temperature	Ground tests, flight	Sensor output, downlink observation	Critical
SC-05	Telemetry is correctly formatted and sent	All phases (exc. RS)	Telemetry logs, no bandwidth issues	Medium
SC-06	Experiment begins at uplink signal reception	Microgravity	Telemetry logs, microscope LEDs	Critical
SC-07	No unrecoverable errors	All phases	Error logs	Medium
SC-08	No error shall crash the software	All phases	Error logs, overall system health	Critical
SC-09	Microscope imaging is correctly collected throughout	All flight phases	Telemetry logs, microscope status	High
SC-10	An unexpected crash shall not corrupt previously collected data	All phases	Error logs, check stored data	High
SC-11	Software must correctly receive LO/SOE/SODS	All phases	Experiment status, logged files	Critical

Fig. 2 Success criteria list with verifications and criticality

3 – Software operational concept

3.1 Concept of operations

This segment explains how the software will act throughout the mission. The software will be developed in a way that maximizes autonomous functionality. Key characteristics of the software will be:

- Three critical ground commands will be sent during the mission via the RXSM to do the following operations: LO, SOE (introduce Dictyostelium cells and medium to microscope observation area), SODS (start recording and storing all data). A fourth critical signal will be sent from the ground station to enable or disable RS.
- The software will have a deterministic execution regarding error handling and the startup. It will have to transition to the appropriate mode depending on the system's health and use-case.
- The software is set to be fault recoverable but not fault tolerant. All potential errors will, to the best extent possible, be targeted during development, so that the least number of errors occur during flight. The code will have to be able to recover from any faults that occur.

The following schematic shows a simplified overview of the software architecture:

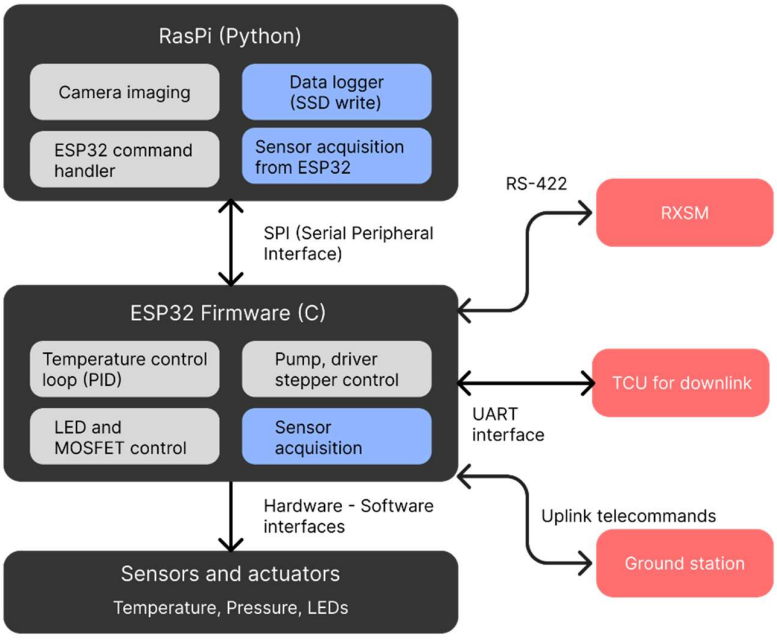

Fig. 3 Software architecture overview

3.2 Software modes and state transitions

3.2.1 Modes

Mode	Abbreviation	Use/Scope
Startup	SU	Boot experience, software begins running
Test	T	Verify system health right after startup
Normal ground	NG	Software is functional on the ground
Normal flight	NF	Software is functional during flight
Normal experiment	NE	Software is functional and experiment is ongoing
Degraded	D	Software operates only a part of its actions because some system elements failed
Safe	S	Software experienced a critical error and stopped functioning to preserve and secure existing data

Fig. 4 Definition of operational modes

Mode	Activation
SU	Once the EM is connected to electrical power
T	Once the startup phase is complete
NG	Once the test phase is complete
NF	After the LO signal is received, & after SOE off
NE	After the SOE signal is received, turns off when SOE is deactivated
D	In case of non-critical errors detected
S	In case of critical errors

Fig. 5 Activation criteria to transition into a given operational mode

3.2.2 Operational scenarios

On the ground, there will be a command that allows the ground station team to deactivate telemetry to respect the RS phase. There is no dedicated mode for RS since the system is disconnected from power. Once it restarts, it enters SU mode again.

- **SU:** During the module startup, the main software functions begin working throughout the hardware.
- **T:** Once the system has booted, a series of tests begin to ensure that the system's health is flight-worthy. The tests include:

- Verify that hardware is functioning (CPU, accessible memory, peripherals and interfaces)
- Verify that sensors are active and that they transmit coherent data
- Verify that there is power running in the system and that the system clock can run correctly (important for mission timestamps)
- Verify that there is available storage space, ability to upload data into storage (there is a very large amount of storage, but a corrupted data card could quickly invalidate this)

- **NG:** After and if the tests are validated, the normal ground mode starts. No major operations occur; the system is GO for launch. Temperature control is active, and data storage starts at SODS shortly before launch.
- **NF:** Activated at the reception of the LO signal. Data logging frequency is increased, and temperature control continues. Later, after the SOE signal is turned off, NF mode is used for the remainder of the flight and descent
- **NE:** Activated once the SOE signal is activated. This starts the experiment operations and continues with the previous tasks. Only operational during the experiment phase.
- **D:** Degraded mode begins if non-critical errors occur and aims to avoid any further cascade of errors (graceful degradation). It is not activated by minor errors: watchdogs verify that the error is serious enough before entering D mode.
- **S:** Safe mode begins if serious system failures occur. The system cannot leave this mode if it begins. Data is secured and preserved and all major operations stop working.

Before LO, if the system enters D mode, it is considered that the flight can still occur. However, if S mode begins, the EM will not be operational during the flight. After LO, the system regularly checks its overall health and enters one of these two modes if necessary (see fig. 6).

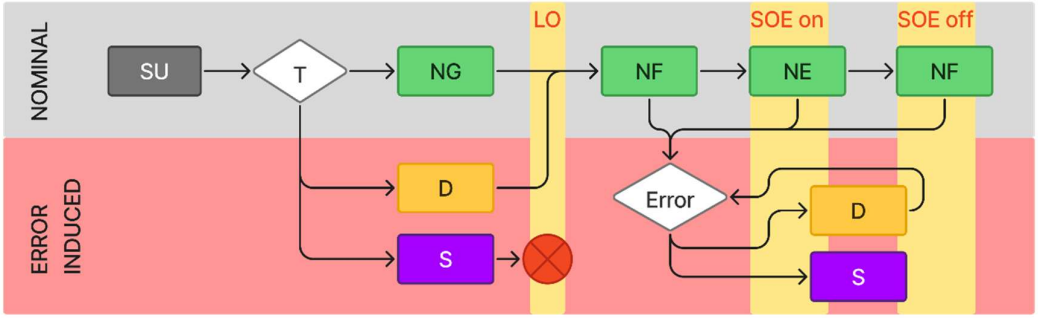

Fig. 6 Nominal and error-induced state-transitions

Near the end of the flight, SODS is turned off and the system awaits power cutoff. All data is stored in memory by this phase, and the EM runs the NF mode until T+600s.

3.3 Interfaces

Interfaces with fixed sampling rates send data from the sensors to the ESP-32. Digital signals are sent from the flight computers to control the stepper motors, the LEDs and the temperature regulation unit. A Serial Peripheral Interface (SPI) that connects the two flight computers will coordinate experiment execution. A UART interface is used between the ESP32 and the TCU. An RS-422 interface is used between the EM and the RXSM for uplink telecommands from the ground station before LO.

4 – Data flow and management

4.1 Sources and acquisition

This section describes data sources and explains their purpose, and when said data is to be collected/received.

Data ID	Source	Data type	Data purpose	Mission phase
D-01	ESP-32	Error logs	Describe a detected error	All phases
D-02	BME sensors	Environmental	Characterize experiment environment	All phases
D-03	Gyroscope/ accelerometer	Inertial	Phase detection and motion data	Launch, ascent, microgravity
D-04	Digital temperature sensor	Environmental	Closed-loop temperature control	All phases
D-05	RasPi Camera V2	Image	Visualization of experiment	Throughout the flight
D-06	RasPi	Metadata	Data context and mission logs	All phases
D-07	SM	Experiment signals (LO, SOE, SODS)	Initiate respective mission phases from SM	LO and microgravity
D-08	Ground station to TCU	Uplink commands	Used before LO to control RS	Pre-launch
D-09	TCU to ground station	Systems data	Flight status downlink	All phases

Fig. 7 Data sources, type and purpose

4.2 Processing and storage, data flow

Data priority and hierarchy are as follows:

1. Imaging data, experiment visualization
2. Experiment-crucial sensor data and experiment module temperature control
3. Status data and housekeeping
4. Diagnostics logging and debugging

Acquired data is subject to validation checks to detect errors, sensor malfunctions, and to block data that could compromise the system or data analysis.

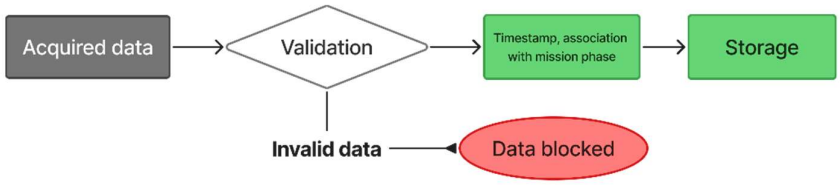

Fig. 8 Validation of acquired data, storage pipeline

Imaging data will be stored as a video into the M.2 storage unit, and all sensor or system data will be logged into .csv files. The .csv files are split into different categories: metadata from system state, metadata from sensors, error logs, telemetry output and camera status and information. Video and .csv files will be fixed to contain a set number of frames and lines respectively, and new ones will be created once the previous one is full. This ensures that all of the recorded data is split into multiple files, so that if one file is corrupted, flight data as a whole is not lost entirely.

4.3 Data output and telemetry

All the data that will be sent as a telemetry downlink to the ground station is data that is also logged in the EM as metadata. This ensures that if the telemetry downlink bits are lost, there is no crucial data that will be lost since it is also stored onboard. The telemetry will send many types of data to the ground team so that the experiment can be monitored in real time. No images from the microscope will be sent since the downlink packet size is very limited, but receiving other data such as SSD status, temperatures, software states and camera statuses will be sufficient to know if the experiment is unfolding correctly.

If the TV channels are available (depending on other teams' needs), the experiment will be able to be monitored in real time on the ground station. This means that the live video can also be saved by the ground station and will serve a backup of the video recorded onboard.

LEDs will be used during ground testing and system startups to have visual confirmation that the system has been booted correctly.

After the launch, data will be recovered by extracting it from the storage cards and from the flight computers. This will have to be done with caution to avoid corrupting or losing the files. All data recovery can be done offline. The recovered videos will show the results of the experiment. All the other metadata can be used to design graphs that can be correlated to experiment phases and performance.

5 – Software constraints and requirements overview

5.1 Physical and environmental constraints

External constraints imposed by the hardware and the mission environment will bring some limits to the system's capabilities. The software is designed to operate well within the processing limits of the flight computers, to ensure lightweight execution of the code. The software will account for transient faults and will need to detect them to contain the issue and to autonomously switch to a recovery state.

Vibrations, quick temperature variations and radiation effects could have an impact on the system, so tolerance levels will have to be set. Since there is no in-flight access to any onboard systems, autonomy will have to be at the center of developing how the software must act overall.

5.2 Operational constraints

The software operates under the constraint of a semi-autonomous mission profile. LO, SOE and SODS signals are sent from the RXSM, but all the other operational behavior is pre-defined prior to launch, and the software must run reliably within a fixed mission timeline.

6 – Verification and validation overview

6.1 Verification concept

Software verification will be performed using unit tests during code development, integration tests in a high-fidelity simulation environment, and formal code inspections.

6.2 Validation concept

Validation will be performed by executing representative mission scenarios in a controlled simulation environment, measuring performance against defined mission success criteria, and obtaining operational approval from the REXUS/BEXUS team.

7 – Abbreviations and notes

EM	Experiment module
LO	Lift-off (signal)
SOE	Start of experiment (signal)
SODS	Start of data storage (signal)
TCU	Telemetry central unit
T	Time before and after launch noted with + or –
RXSM	REXUS Service module
RS	Radio silence

[END OF DOCUMENT]